Examen parcial de teoría de SO

Instrucciones para el examen:

- **Justifica todas tus respuestas del examen.** Las respuestas no justificadas se considerarán erróneas.
- Podéis consultar el man instalado en el PC **pero ningún otro material.**
- **Utiliza el fichero template** que os damos junto con este enunciado para responder a las preguntas del examen y **entrégalo a través del racó** (solo un fichero).
- Utiliza un editor **en modo texto**.
- **Utiliza solo el espacio** que hay **reservado** para cada respuesta.
- Si no puedes responder una pregunta deja en blanco el espacio reservado para esa pregunta (no borres ninguna pregunta).

Preguntas Cortas (*2 puntos*)

1) ***(0,5 puntos)*** Indica dos llamadas a sistema que puedan causar que un proceso cambie de estado RUN a estado BLOCKED, e indica qué circunstancias deben ocurrir para que el bloqueo ocurra.
2) ***(0,5 puntos)*** En un algoritmo de planificación Round Robin, teniendo en cuenta que tenemos 3 procesos activos, y un quantum de 10 ms, ¿es posible que un proceso llegue a ejecutarse dos veces en menos de 30 ms? ¿En qué circunstancias?
3) ***(0,5 puntos)*** Explica de forma precisa cuál es la función del argumento sa_mask del struct sigaction.
4) ***(0,5 puntos)*** ¿Cuáles son los tres tipos de eventos que pueden causar que se entre al kernel?

Pregunta Procesos (*4 puntos*)

La figura 2 1 muestra el código del programa CTprocs.c. Se omite el control de errores para facilitar la legibilidad del código. Ponemos en ejecución este programa desde el Shell de la siguiente forma:

```
%./CTprocs 3
```

```c
int main (int argc, char **argv)
{
  int n = atoi (argv[1]);
  int i = 0, r, status = -1,ret;
  char buff[256];
  while (i < n) {
      r = fork ();
      i++;
    }
  i = 0;
  while (i < n) {
      if (r == 0) execlp ("./hola", "hola", (char *) 0);
      i++;
  }
  while (waitpid (-1, &status, 0) > 0){
      ret=sprintf (buff,"status: %d->%d\n", getpid (), status);
      write(1,buff,ret);
  }
  exit (0);
}
```

```c
int main()
{
    int ret;
    char buff[256];
    ret=sprintf(buff,"HOLA!!! --- %d\n", getpid());
    write(1,buff,ret);
    exit(0);
}
```

figura 1 Código de CTprocs.c (izquierda) y hola.c (derecha)

1) ***(0,75 puntos)*** Describe la jerarquía de procesos que se crea al ejecutar, cuando todos los programas estén en la línea 11. Identifica los procesos, para preguntas posteriores: llama P0 al proceso inicial. ¿La ejecución de la jerarquía es secuencial o concurrente?
2) ***(0,5 puntos)*** ¿Qué procesos de tu jerarquía ejecutan el programa "hola"?
3) ***(0,5 puntos)*** ¿Cuántas veces se ejecuta la línea 16, en el conjunto de todos los procesos? Justifica indicando qué procesos la ejecutan y cuantas veces lo hacen.
4) ***(0,75 puntos)*** Describe semánticamente el valor de la variable status en la línea 16 para todas las veces que se ejecuta.
5) ***(0,75 puntos)*** Si cambiamos la línea 12 por

```
if (r > 0) execlp ("./hola", "hola", (char *) 0);
```

¿Cuántas veces en toda la jerarquía de procesos se ejecutará el sprintf de la línea 16?
6) ***(0,75 puntos)*** Sustituye el bucle de las lineas 15 a 18, para que manteniendo la funcionalidad, el waitpid no sea bloqueante**.**

Pregunta Signals (*4 puntos*)

La figura 2 muestra el código del programa test_signal.c. Se omite el control de errores para facilitar la legibilidad del código. Ponemos en ejecución este programa desde el Shell. Sabemos que la gestión que haga de los signals es la especificada en el propio código y que no va a recibir ningún signal desde fuera del código.

```
1  trat_signal(int s) {
2      char buf[80];
3      sprintf(buf,"Signal recibido %d\n", s);
4      write(1, buf, strlen(buf));
5      exit(0);
6  }
7
8  int main () {
9      sigset_t pmask ;
10     int ret;
11     struct sigaction trat;
12         sigfillset(&pmask);
13         sigprocmask(SIG_SETMASK, &pmask,NULL);
14         trat.sa_flags=0;
15         sigemptyset(&trat.sa_mask);
16         trat.sa_handler=trat_signal;
17         ret = fork();
18         if (ret == 0) {
19             alarm (2) ;
20             kill (getppid(), SIGUSR1);
21             while(1){
22             }
23         } else{
24             sigaction(SIGUSR1, &trat, NULL);
25             sigdelset(&pmask, SIGUSR1);
26             sigprocmask(SIG_SETMASK, &pmask, NULL);
27             execlp("./bucle", "bucle", NULL);
28         }
29 }
```

```
1 int main() {
2     while(1) {
3     }
4 }
```

figura 2: Código de test_signal.c (izquierda) y bucle.c (derecha)

Supongamos que ninguna llamada a sistema devuelve error. Contesta razonadamente a las siguientes preguntas.

1) ***(0,5 puntos)*** ¿Qué signals recibe cada proceso?
2) ***(0,5 puntos)*** Para cada proceso considera el último valor que toma la máscara de signals bloqueados de su PCB. Para cada proceso: ¿en qué línea de código se pone ese valor? ¿qué signals bloquea esa línea?
3) ***(0,75 puntos)*** ¿Qué signals trata cada proceso? ¿Con qué tratamiento lo hace?
4) ***(0,75 puntos)*** Supón que movemos la línea 19 (alarm(2)) justo antes del fork. ¿Qué signals recibe ahora cada proceso?

5) ***(1,5 puntos)*** Si ejecutamos varias veces este programa vemos que lo que se muestra por pantalla es diferente cada vez. Unas veces aparece el mensaje “Signal recibido 10” y otras veces aparece el mensaje “User defined signal 1”.
   a. ***(0,75 puntos***) Para las veces que aparece el mensaje “User defined signal 1”, ¿quién está escribiendo ese mensaje? ¿puedes saber en qué punto del código (o en qué condiciones de la ejecución) estaba el proceso que ha recibido el signal que ha provocado este mensaje?
   b. ***(0,75 puntos)*** Se quiere garantizar que siempre se va a ver por pantalla únicamente el mensaje “Signal recibido 10”. Para ello se propone reemplazar la línea 26 por lo siguiente: sigsuspend(&pmask). ¿Se conseguirá ese objetivo?